



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



PREVIO N° 07

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 09/04/2026

CALIFICACIÓN: _____

Previo 7: Iluminación:

1.- Explicar los valores de *con*, *lin*, *exp* cómo influyen en las luces puntuales y spotlight y demostrar con capturas de pantalla los diferentes fenómenos de la luz al variar dichos valores.

R= Los valores de atenuación constante (K_c), lineal (K_l) y cuadrática/exponencial (K_q) determinan cómo disminuye la intensidad de una luz conforme se aleja de su origen. La fórmula matemática estándar utilizada en el *Fragment Shader* es:

$$f_{att} = \frac{1.0}{K_c + K_l \cdot d + K_q \cdot d^2}$$

Donde:

- **Constante (K_c):** Generalmente se mantiene en 1.0. Su función es asegurar que el denominador nunca sea menor que uno, evitando que la intensidad de la luz se dispare a valores astronómicos cuando la distancia (d) es muy pequeña (de Vries, 2020).
- **Lineal (K_l):** Reduce la intensidad de la luz de forma proporcional a la distancia. Se utiliza para simular una pérdida de energía moderada en el espacio.
- **Cuadrática/Exponencial (K_q):** Reduce la intensidad de forma cuadrática respecto a la distancia (d^2). Este valor es el que más influye en el realismo, ya que simula cómo la luz se dispersa físicamente en un área esférica (*Constant-Linear-Quadratic Falloff* - Valve Developer Community, n.d.) (LearnOpenGL, s.f.). (*Tutorial 8 : Basic Shading*, n.d.)

Casos para la Luz del Helicóptero (spotLights)

Caso de Iluminación "Infinita" (Sin pérdida de energía):

- Valores: 1.0f, 0.0f, 0.0f
- Efecto: El haz de luz amarillo llegará al piso con la misma intensidad sin importar la altura del helicóptero.



Luz con posición cercana



Luz con posición lejana

Caso de Atenuación Realista (Luz media):

- Valores: 1.0f, 0.045f, 0.0075f
- Efecto: La luz tendrá un alcance considerable, pero tiene un degradado suave. El centro será brillante y las orillas del haz se perderán gradualmente en la oscuridad del piso.



Luz con posición cercana



Luz con posición lejana

Caso de Alcance Corto (Luz de bolsillo):

- Valores: $1.0f$, $0.22f$, $0.20f$
- Efecto: La luz se volverá muy débil muy rápido. Cuando sube el helicóptero, el haz apenas será visible en el suelo.



Luz con posición cercana

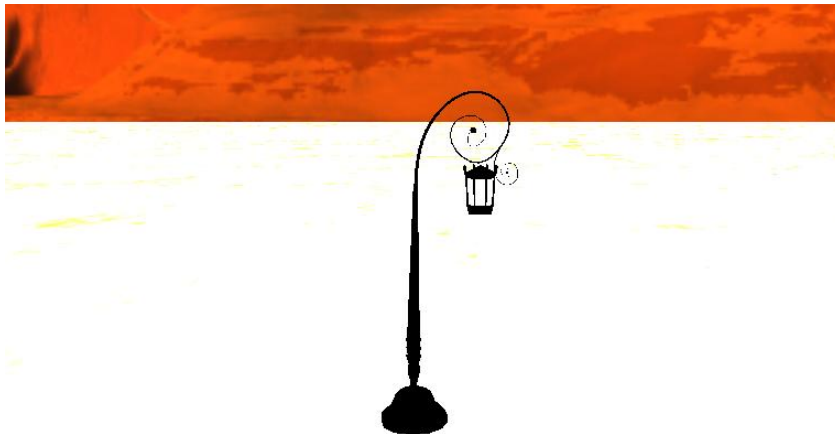


Luz con posición lejana

Casos para la Luz de la Farola (pointLights)

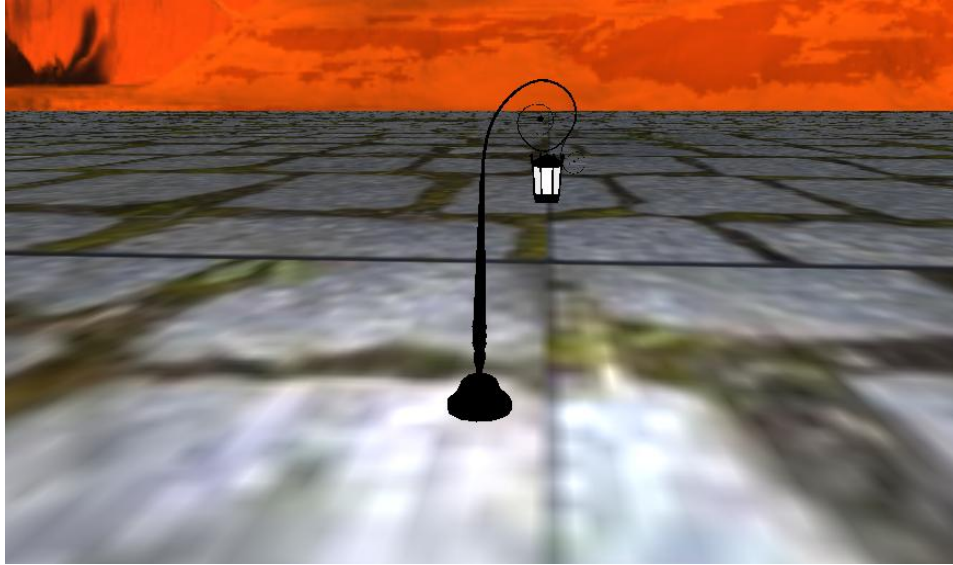
Caso de "Explosión" Lumínica (Saturación):

- Valores: $0.1f$, $0.0f$, $0.0f$
- Efecto: Al ser la constante menor a 1, el denominador es muy pequeño y el brillo se dispara. La zona alrededor de la farola se verá blanca sólida (pérdida total de detalle por sobreexposición).



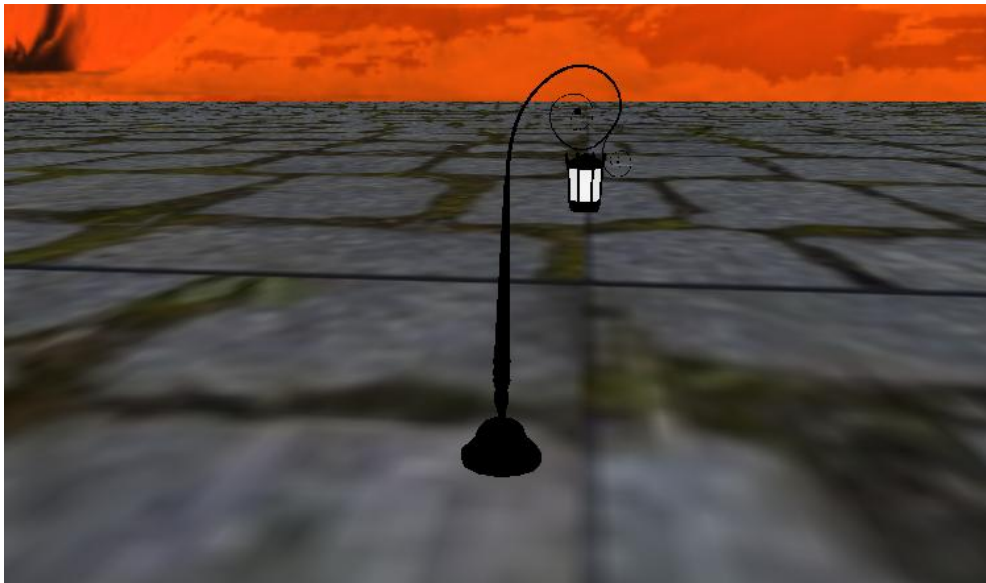
Caso de Atenuación Estándar:

- Valores: $1.0f$, $0.09f$, $0.032f$
- Efecto: Es el valor equilibrado para una luz que cubre aproximadamente 50 unidades de distancia.



Caso de Oscuridad Casi Total:

- Valores: $1.0f$, $0.7f$, $1.8f$
- Efecto: La luz blanca se verá como una pequeña chispa que no ilumina prácticamente nada del entorno.



2.- ¿Cómo haces para que en tiempo de ejecución puedas elegir entre 2 arreglos de 4 luces puntuales en orden diferente, es decir: en el arreglo 1 tienes las luces verde, azul, roja, blanca y en el arreglo dos tienes a las luces blanca, verde, azul, roja?

R= Para alternar entre diferentes configuraciones de colores en un arreglo de luces (como cambiar de Verde-Azul-Rojo-Blanca a Blanca-Verde-Azul-Rojo) durante la ejecución, se utiliza una **variable de estado o bandera (flag)** dentro del bucle principal (de Vries, 2020).

Al detectar la presión de una tecla mediante `glfwGetKey`, se actualizan los valores de color de los objetos `PointLight` dentro del arreglo global `pointLights[ ]` antes de enviarlos nuevamente al shader mediante la función `SetPointLights` (Project, n.d.)

Debido a que el shader recibe la información de las luces en cada cuadro (frame), el cambio es instantáneo para el usuario. Esto es más eficiente que tener dos arreglos físicos pesados; basta con reasignar los atributos de color de las luces existentes (Bock, n.d.)

Conclusión

Al realizar este cuestionario previo y las pruebas de iluminación, pude comprender mejor la importancia de los parámetros de atenuación en las fuentes de luz puntual y *spotlight*. Entendí que la luz no solo requiere una posición y un color, sino la aplicación correcta de la **ecuación de atenuación**, la cual es fundamental para que la energía de la luz decaiga de forma realista y se eviten efectos irreales como la "iluminación infinita" que vimos en la sesión de laboratorio.

Además, mediante la investigación de la pregunta 2, comprendí que la eficiencia del programa no depende de saturar el sistema con una gran cantidad de luces simultáneas, sino en la capacidad de **reutilizar un número menor de luces**. En lugar de sobrecargar la memoria y el Shader con múltiples arreglos físicos estáticos, es mejor emplear una variable de estado que permita alternar el contenido de un solo arreglo en tiempo de ejecución. Detectando eventos de teclado para actualizar los atributos de las luces existentes, y así poder cubrir diferentes combinaciones de iluminación sin incrementar el número de luces y así poder mejorar el rendimiento.

Bibliografía

- LearnOpenGL - Basic Lighting. (n.d.). <https://learnopengl.com/Lighting/Basic-Lighting>
- LearnOpenGL - Light casters. (n.d.). <https://learnopengl.com/Lighting/Light-casters>
- Project, O. T. (n.d.). Multiple lights - OpenTK. OpenTK.
<https://opentk.net/learn/chapter2/6-multiple-lights.html>
- Bock, D. (n.d.). CSC 231 - Computer Graphics i.
<https://remy.parkland.edu/~dbock/Class/csc231/Lecture/OpenGLLighting.html>
- Constant-Linear-Quadratic falloff - Valve Developer Community. (n.d.).
https://developer.valvesoftware.com/wiki/Constant-Linear-Quadratic_Falloff
- Tutorial 8 : Basic shading. (n.d.). <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-8-basic-shading/>